

MULUNGUSHI UNIVERSITY

School of Engineering and Technology

ICT461 – WEB SYSTEMS AND TECHNOLOGY

Student: Beaty Chilingila

Student ID: 202300995

Course: Web Systems

Lecturer: Mr. Nyirenda

1. Objective

Task 2 Part 1 requires the six API routes of the course registration portal to be tested with cURL before any browser connection is attempted, so that the raw HTTP contract (status codes, headers and body) can be verified independently of client-side code. This report presents that evidence and explains three specific requirements from the brief:

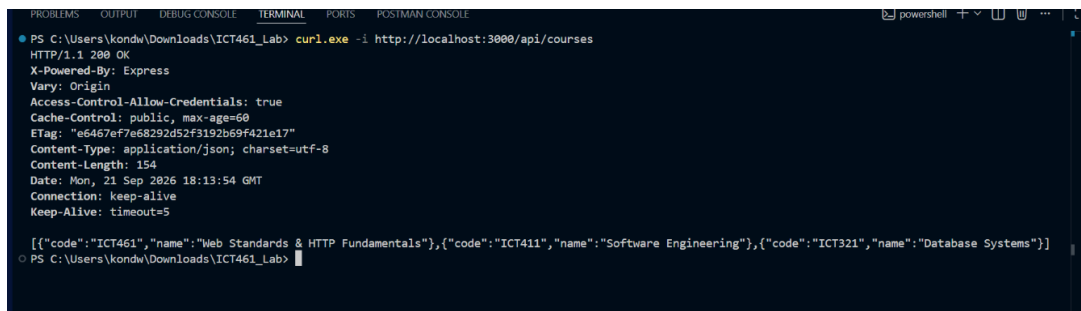
- Check response.ok before treating a response as successful, rather than assuming any response that arrives without a network error is a success.
- Never parse a 204 No Content response as JSON, since it has no body.
- Repeat POST, PUT and DELETE requests and explain why idempotency concerns the effect the request has on server state, not whether the status code is identical on every repetition.

All evidence below was captured with curl.exe -i against the API running on http://localhost:3000, before the CORS/browser step (step 8) of the lab was attempted.

2. Route-by-Route cURL Evidence

GET /api/courses — success case

The base course list is returned with 200 OK. The response also carries an ETag and Cache-Control: public, max-age=60, which are exercised in the caching test below.

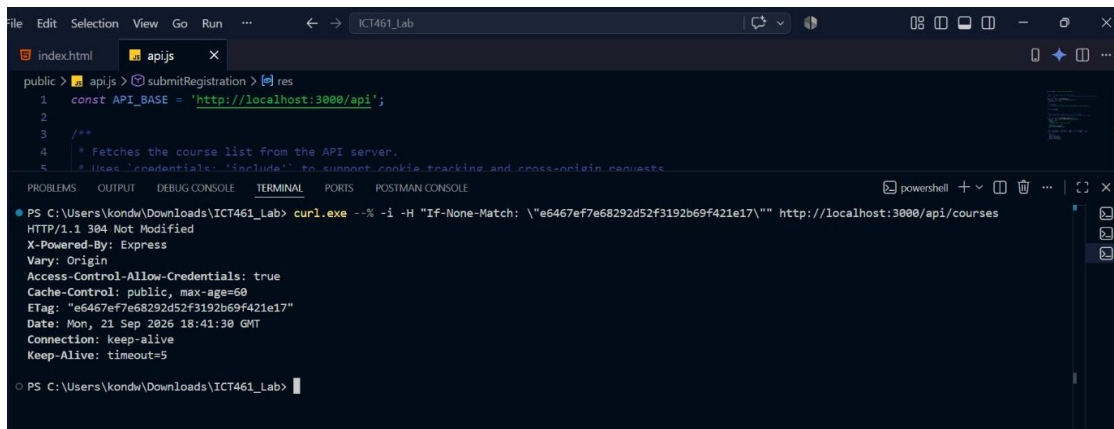


```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i http://localhost:3000/api/courses
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: public, max-age=60
ETag: "e6467ef7e68292d52f3192b69f421e17"
Content-Type: application/json; charset=utf-8
Content-Length: 154
Date: Mon, 21 Sep 2026 18:13:54 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"code":"ICT461","name":"Web Standards & HTTP Fundamentals"}, {"code":"ICT411","name":"Software Engineering"}, {"code":"ICT321","name":"Database Systems"}]
PS C:\Users\kondw\Downloads\ICT461_Lab>
```

Cache revalidation with If-None-Match

Sending the ETag value back in an If-None-Match header on a repeat GET produces 304 Not Modified with no response body, confirming the server is revalidating rather than resending the unchanged representation.



```
public > apijs > submitRegistration > res
1 const API_BASE = 'http://localhost:3000/api';
2
3 /**
4  * Fetches the course list from the API server.
5  * Uses credentials: 'include' to support cookie tracking and cross-origin requests
6  */
7
8 // ...

PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe --% -i -H "If-None-Match: 'e6467ef7e68292d52f3192b69f421e17'" http://localhost:3000/api/courses
HTTP/1.1 304 Not Modified
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: public, max-age=60
ETag: "e6467ef7e68292d52f3192b69f421e17"
Date: Mon, 21 Sep 2026 18:41:38 GMT
Connection: keep-alive
Keep-Alive: timeout=5

PS C:\Users\kondw\Downloads\ICT461_Lab>
```

POST /api/registrations — create and duplicate rejection

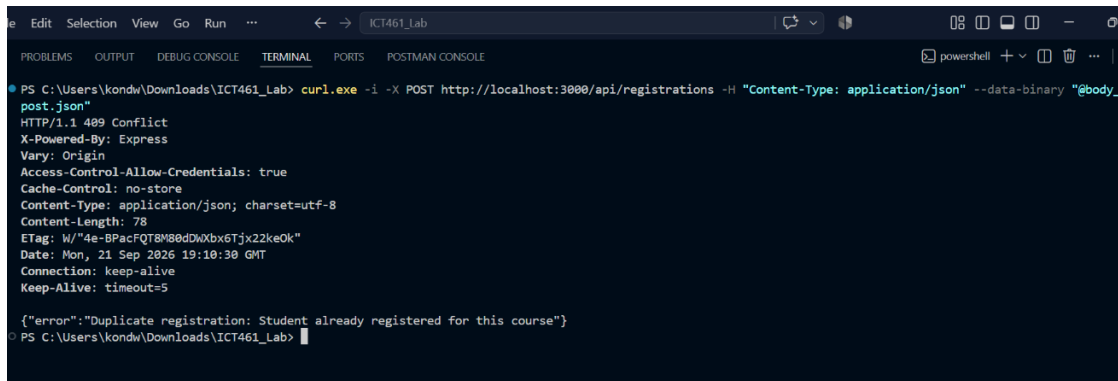
The first POST with a new student/course combination succeeds with 201 Created, a Location header pointing at the new resource, and Cache-Control: no-store (registration responses are explicitly not cached).



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X POST http://localhost:3000/api/registrations -H "Content-Type: application/json" --data-binary "@body_post.json"
HTTP/1.1 201 Created
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Location: /api/registrations/1
Content-Type: application/json; charset=utf-8
Content-Length: 105
ETag: W/"69-DFM76zrDxAU-rvDSNuDOSdQzVS9w"
Date: Mon, 21 Sep 2026 19:07:30 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"studentName":"Alice","studentId":"2024001","programme":"BSc Computer Science","course":"ICT461"}
```

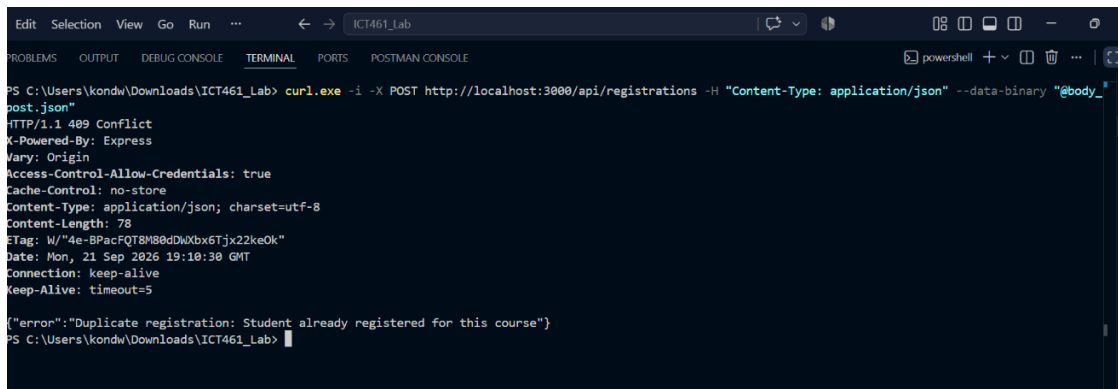
Repeating the identical POST — same student ID and course — is rejected with 409 Conflict and an explicit "Duplicate registration" error body, on both the first repeat and a second repeat afterwards.



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X POST http://localhost:3000/api/registrations -H "Content-Type: application/json" --data-binary "@body_post.json"
HTTP/1.1 409 Conflict
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 78
ETag: W/"4e-BPacFQT8M80dDkXbx6Tjx22keOk"
Date: Mon, 21 Sep 2026 19:10:30 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Duplicate registration: Student already registered for this course"}
```

Figure 4a. Repeat POST of the same registration → 409 Conflict.



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X POST http://localhost:3000/api/registrations -H "Content-Type: application/json" --data-binary "@body_post.json"
HTTP/1.1 409 Conflict
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 78
ETag: W/"4e-BPacFQT8M80dDkXbx6Tjx22keOk"
Date: Mon, 21 Sep 2026 19:10:30 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Duplicate registration: Student already registered for this course"}
```

Figure 4b. Repeating the same POST again → 409 Conflict a second time, same error body.

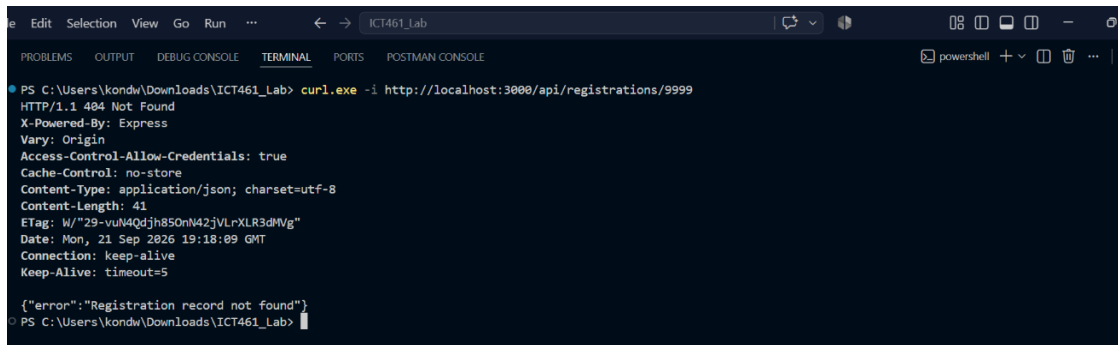
GET /api/registrations/:id — found and not found



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i http://localhost:3000/api/registrations/1
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 105
ETag: W/"69-DFM76zrDxAUrvDSNuD05dQzV59w"
Date: Mon, 21 Sep 2026 19:17:01 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"studentName":"Alice","studentId":"2024001","programme":"BSc Computer Science","course":"ICT461"}
```

Figure 5. *GET /api/registrations/1 → 200 OK, the stored Alice record.*

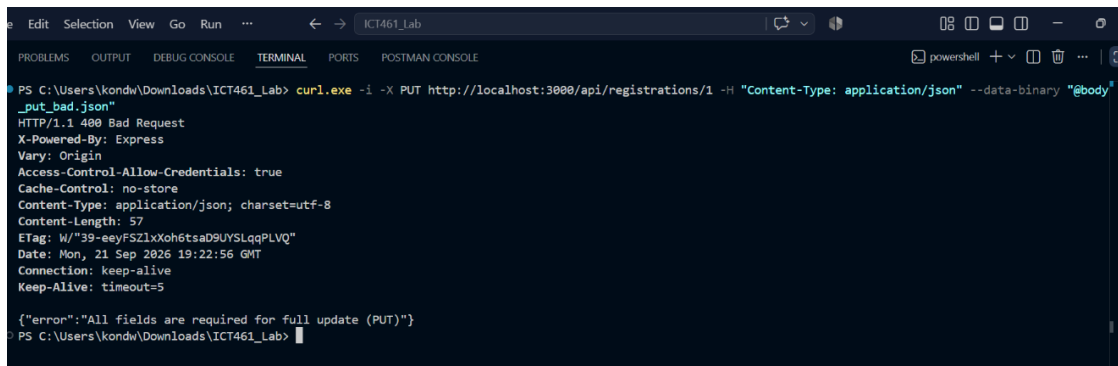


```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i http://localhost:3000/api/registrations/9999
HTTP/1.1 404 Not Found
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 41
ETag: W/"29-vuN4Qdjh85OnM42jVLR3dWVg"
Date: Mon, 21 Sep 2026 19:18:09 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Registration record not found"}
```

PUT /api/registrations:id — full replace

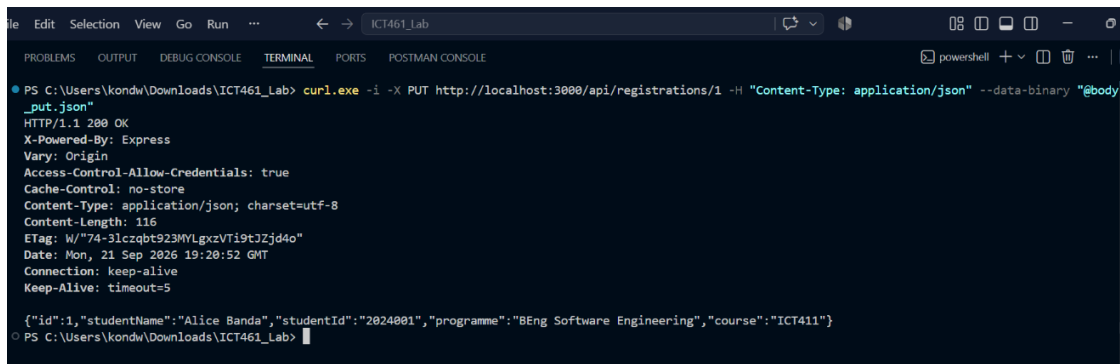
PUT enforces that every required field is present. Submitting a partial body is rejected with 400 Bad Request before any record is touched; submitting the complete replacement body succeeds with 200 OK and returns the fully replaced record.



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X PUT http://localhost:3000/api/registrations/1 -H "Content-Type: application/json" --data-binary "@body_put_bad.json"
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 57
ETag: W/"39-eeYFSZ1xXoh6tsaD9UYSLqQPLVQ"
Date: Mon, 21 Sep 2026 19:22:56 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"All fields are required for full update (PUT)"}
PS C:\Users\kondw\Downloads\ICT461_Lab>
```

Figure 7. *PUT with a field missing → 400 Bad Request, "All fields are required for full update (PUT)".*



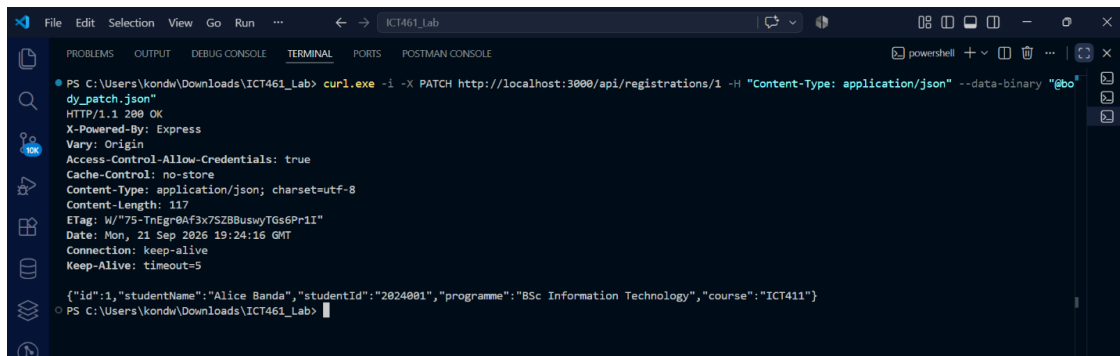
```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X PUT http://localhost:3000/api/registrations/1 -H "Content-Type: application/json" --data-binary "@body_put.json"
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 116
ETag: W/"74-31czqbt923MYLgxzVti9tJZjd4o"
Date: Mon, 21 Sep 2026 19:20:52 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"studentName":"Alice Banda","studentId":"2024001","programme":"BEng Software Engineering","course":"ICT411"}
```

Figure 8. PUT with the complete body → 200 OK, record fully replaced (name, programme and course updated).

2.6 PATCH /api/registrations/:id — partial update of programme only

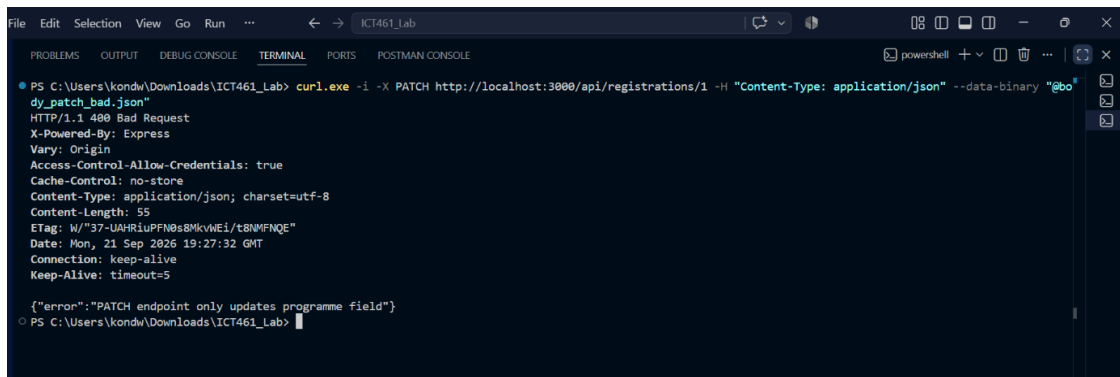
PATCH is scoped to the programme field only. A valid programme change succeeds with 200 OK; attempting to change a different field through PATCH is rejected with 400 Bad Request.



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X PATCH http://localhost:3000/api/registrations/1 -H "Content-Type: application/json" --data-binary "@body_dy_patch.json"
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 117
ETag: W/"75-TnEgr0Af3x7S2B8uswyTGs6Pr1I"
Date: Mon, 21 Sep 2026 19:24:16 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"studentName":"Alice Banda","studentId":"2024001","programme":"BSc Information Technology","course":"ICT411"}
```

Figure 9. PATCH { programme } → 200 OK, only the programme field changes.



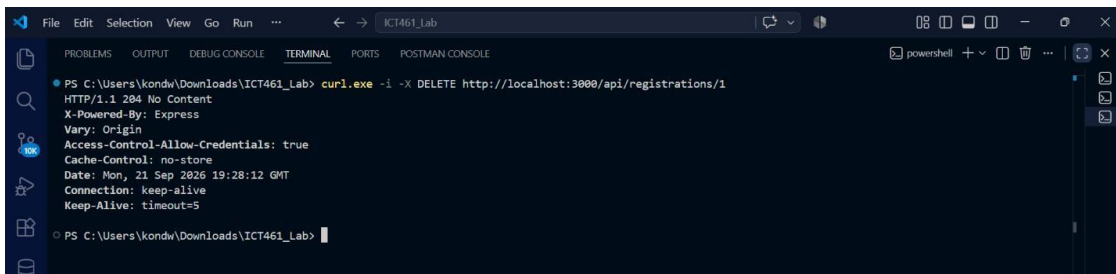
```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X PATCH http://localhost:3000/api/registrations/1 -H "Content-Type: application/json" --data-binary "@body_dy_patch_bad.json"
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 55
ETag: W/"37-UAHRIuPFN0sBMkvNEI/tBwWFNQe"
Date: Mon, 21 Sep 2026 19:27:32 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"PATCH endpoint only updates programme field"}
```

Figure 10. PATCH attempting an out-of-scope field → 400 Bad Request, "PATCH endpoint only updates programme field".

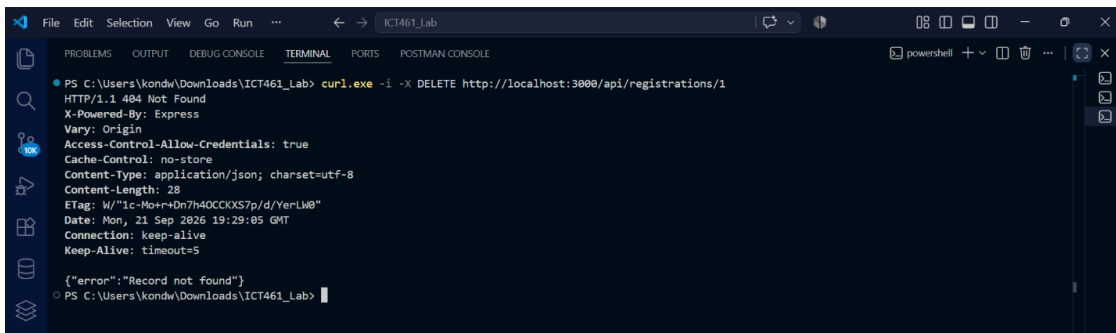
2.7 DELETE /api/registrations/:id — remove and repeat

The first DELETE removes the record and returns 204 No Content with an empty body, as required. Repeating the same DELETE against the now-missing record returns 404 Not Found rather than a second 204.



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X DELETE http://localhost:3000/api/registrations/1
HTTP/1.1 204 No Content
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Date: Mon, 21 Sep 2026 19:28:12 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Figure 11. DELETE /api/registrations/1 (first call) → 204 No Content, empty body.



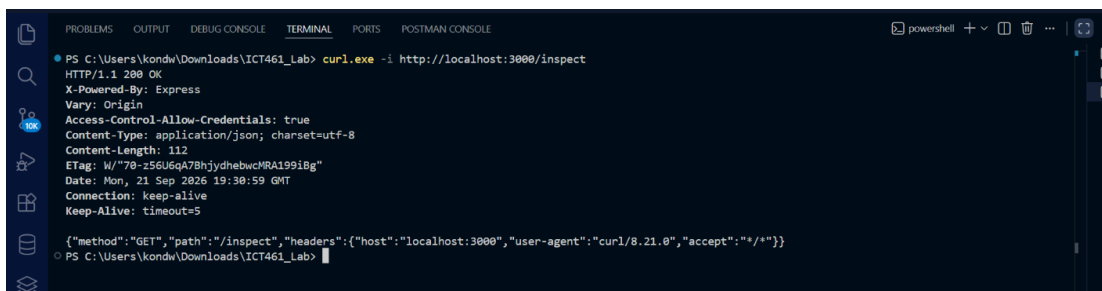
```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i -X DELETE http://localhost:3000/api/registrations/1
HTTP/1.1 404 Not Found
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Cache-Control: no-store
Content-Type: application/json; charset=utf-8
Content-Length: 28
ETag: W/"1c-Mo+r+Dn7h40CCX57p/d/YerLW0"
Date: Mon, 21 Sep 2026 19:29:05 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Record not found"}
```

Figure 12. Repeat DELETE /api/registrations/1 → 404 Not Found, "Record not found".

2.8 Supporting diagnostic routes

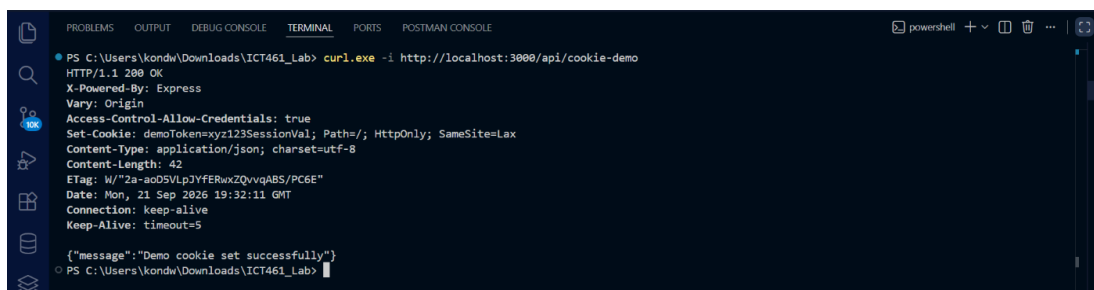
The /inspect route echoes method, path and headers for later comparison of Accept vs Content-Type, and the /cookie-demo route sets a non-sensitive HttpOnly cookie used in the later state/security investigation.



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i http://localhost:3000/inspect
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Content-Type: application/json; charset=utf-8
Content-Length: 112
ETag: W/"70-56U6qA79hjydehbcwRA199i8g"
Date: Mon, 21 Sep 2026 19:30:59 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"method":"GET","path":"/inspect","headers":{"host":"localhost:3000","user-agent":"curl/8.21.0","accept":"*//*"}}
```

Figure 13. GET /inspect → 200 OK, echoes method, path and request headers.



```
PS C:\Users\kondw\Downloads\ICT461_Lab> curl.exe -i http://localhost:3000/api/cookie-demo
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Access-Control-Allow-Credentials: true
Set-Cookie: demoToken=xyz123SessionVal; Path=/; HttpOnly; SameSite=Lax
Content-Type: application/json; charset=utf-8
Content-Length: 42
ETag: W/"2a-a0D5VlpJYfERwXZQvvaqABS/PC6E"
Date: Mon, 21 Sep 2026 19:32:11 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"message":"Demo cookie set successfully"}
```

Handling response.ok and the 204body rule

Across every route above, a 2xx status was treated as the only condition for a "successful" response — this is what response.ok checks for on the client side (status in

the 200–299 range), rather than simply checking that `fetch()` resolved without throwing. A resolved fetch promise only means the network request completed; it says nothing about whether the server accepted the request, which is why 400, 404 and 409 responses above all still "succeed" at the network layer while failing `response.ok`.

The DELETE evidence (Figure 11) demonstrates the 204 case directly: the response has Content-Length absent and no body at all. Calling `response.json()` on a 204 would throw, because there is no JSON — or any content — to parse. The correct client-side pattern is to branch on status before attempting to read a body:

```
if (!response.ok) { /* handle 4xx/5xx, read error body if present */ } else if  
(response.status === 204) { /* no body: treat as success, skip parsing */ } else { const  
data = await response.json(); }
```

4. Idempotency: effect on server state, not repeated status codes

The brief specifically asks for an explanation of why idempotency is about the intended effect a request has on the server's stored data, not about whether cURL prints the same status code on every repetition. The evidence gathered above shows all three cases:

4.1 POST — not idempotent

POST is not idempotent by design: each successful call is meant to create a new resource. In Figures 3–4, the first POST changes server state (a new registration, id 1, is created) and returns 201. Repeating the exact same POST does not create a second resource — the duplicate-registration check stops that — but it also does not leave the server in the state a second successful POST would produce. The repeated calls happen to return the same 409 status twice, but that repetition of the status code is incidental to the duplicate-detection logic, not evidence of idempotency: had the duplicate check been absent, each repeat would have created another record and changed the state further every time.

4.2 PUT — idempotent by design

PUT is idempotent: submitting the same complete replacement body more than once should leave the resource in the same final state as submitting it once. In Figure 8, the record is replaced with a specific set of field values. Sending that identical PUT again would return 200 OK with the same resulting record, because the operation is "set these fields to these values", not "add another change" — the server state converges to the same result regardless of how many times the identical request is repeated.

4.3 DELETE — idempotent effect, non-identical status codes

DELETE is the clearest illustration of the brief's point. The first DELETE (Figure 11) removes registration 1 and returns 204 No Content. Repeating the identical DELETE (Figure 12) returns 404 Not Found, not another 204 — the status code changed between the two calls. Despite that, DELETE is still idempotent, because idempotency is defined by the resulting server state, not the status code: after one DELETE, the record is gone;

after two, three or any further identical DELETE calls, the record is still gone. The end state never changes after the first successful call, even though the HTTP status reported to the client does change (204 the first time, 404 thereafter, since the resource no longer exists to be deleted). This is exactly the distinction the task asks for: identical repeated status codes are neither necessary nor sufficient for idempotency — what matters is whether repeating the request keeps producing the same server-side effect.

5. Summary

Method / Route	First call	Repeat call	Idempotent?
GET /api/courses	200 OK	304 Not Modified (revalidated)	Yes — safe & idempotent
POST /api/registrations	201 Created	409 Conflict (both repeats)	No — not idempotent by design
PUT /api/registrations/:id	400 (partial) → 200 (full)	200 OK, same final record	Yes — same state each time
PATCH /api/registrations/:id	200 OK (programme)	400 on out-of-scope field	Yes, for the same valid patch
DELETE /api/registrations/:id	204 No Content	404 Not Found	Yes — end state stays "deleted"

This evidence set satisfies Task 2 Part 1: every route was exercised with cURL ahead of the browser/CORS step, the 204 No Content case was captured without attempting to parse a body, and repeated POST, PUT and DELETE calls were used to show that idempotency is a property of server-side effect, not of the HTTP status code reported on each individual call.